CONNECTLY PLATFORM

System Architecture, Feature Analysis & Engineering Roadmap

Project Type	Architecture	Target Host	Core Stack
Social Networking System	Clean Architecture (4-Layer)	SmarterASP.NET / Monster ASP	ASP.NET Core / EF Core / SignalR

1. Executive Summary & Project Vision

Connectly is a full-featured, real-time social networking web platform built on the ASP.NET Core ecosystem. Designed following enterprise Clean Architecture and the Repository Pattern, the system provides secure identity management, dynamic personal profile spaces, interactive social feeds, asymmetric/bi-directional networking, background automated reminders, and real-time private messaging with message-request safety routing.

The platform guarantees clean separation of concerns, high maintainability, testability, and responsiveness, ensuring seamless deployment to shared Windows/.NET cloud hosting environments (such as Monster ASP / SmarterASP.NET).

2. Clean Architecture Structural Breakdown

The solution strictly enforces the **Dependency Inversion Principle (DIP)** where dependencies point inward toward the core business domain:

1. Connectly.API (Presentation Layer)

HOST & ENTRY POINT

- **Role:** HTTP REST Controllers, SignalR Hubs (Chat & Notifications), JWT Auth Middlewares, Swagger, File Upload Handlers.
- **References:** `Connectly.Application` & `Connectly.Infrastructure` (strictly for DI registration in `Program.cs`).

2. Connectly.Application (Business Core)

USE CASES & RULES

- **Role:** Application services, DTOs, ViewModels, Repository Interfaces (`IUnitOfWork`, `IPostRepository`), AutoMapper profiles, FluentValidators.
- **References:** `Connectly.Domain` only.

3. Connectly.Infrastructure (Adapters & Persistence)

DATA & I/O

- **Role:** EF Core `ApplicationDbContext`, SQL Server configurations, Repository implementations, JWT Generator service, Background Workers (`IHostedService`), Local/Cloud Storage integrations.
- **References:** `Connectly.Application` & `Connectly.Domain`.

4. Connectly.Domain (Enterprise Entities)

CORE ENGINE

- **Role:** Pure Domain Entities (User, Post, Friendship, Message, Notification), Domain Enums (`PostPrivacy`, `FriendshipStatus`, `NotificationType`), Domain Exceptions.
- **References:** *None (Zero external dependencies).*

3. Detailed Feature Specifications

> **Core & Enhanced Feature Breakdown**
> Below is the operational breakdown of both foundational platform modules and high-value architectural additions.

Feature Module	Functional Specification & Business Logic	Technical Implementation Mechanism
JWT Auth & Identity	Secure user registration, login, profile picture upload, bio updates, password hashing, and token issuance with custom user claims.	ASP.NET Core Identity + `JwtBearer` with symmetric token encryption and query-token handler for SignalR.
Friendships & Social Graph	Bi-directional connections with states (`Pending`, `Accepted`, `Blocked`). Viewing mutual friends, user directories, and pending requests.	EF Core composite self-referencing entity (`RequesterId` + `AddresseeId`) with indexed lookup queries.
Post Privacy Selector	Users designate post visibility: `Public` (all users), `FriendsOnly` (accepted friends only), or `OnlyMe` (author only).	`PostPrivacy` enum applied at the SQL query level in EF Core to filter feeds efficiently.
Instant Feed & User Search	Debounced instant discovery for posts (matching content/tags) and user accounts (matching display names and bios).	EF Core `EF.Functions.Like` indexed search with paginated result sets (Skip/Take).
Birthday Reminder Worker	Automated scheduled scan identifying users celebrating birthdays each day and notifying all their accepted friends.	ASP.NET Core `BackgroundService` (IHostedService) running a daily midnight UTC background loop.
Chat Portal & Safety Routing	1-on-1 private messaging. Direct delivery to inbox if users are friends; automatically diverted to "Message Requests" if not friends.	SignalR `ChatHub` with condition-based database flag (`IsRequest = true/false`) based on friendship status.
Read Receipts & Typing Indicators	Live feedback when the recipient is typing, plus visual read receipts (delivered vs. read timestamps) upon opening a conversation.	SignalR ephemeral event broadcasts for typing; persistent DB state updates for message read status.
Real-Time Notifications	Instant toast popups and a persistent notification hub for friend requests, user tags, post interactions, and birthday reminders.	SignalR `NotificationHub` mapped to authenticated `UserId` + SQL notification history table.

4. Developer Prerequisites & Technology Matrix

The following table defines the specific technologies the engineering team must master and correlates each tool with its precise role in the project lifecycle:

Technology	Required Concept Knowledge	Applied Project Process	Hosting Relevance
C# / .NET 8+	OOP, LINQ, Asynchronous programming (`async/await`), Dependency Injection.	Used across all 4 layers for core logic, domain rules, and controllers.	Target runtime on Windows IIS Server.
Entity Framework Core	Fluent API, Code-First Migrations, Composite Keys, Navigation Properties, Query Splitting.	`Connectly.Infrastructure` data persistence, feed queries, and indexing.	Runs against remote MS SQL Server on ASP Monster.
ASP.NET Core Identity & JWT	Claims identity, password hashing, token validation, security algorithms, cookie fallback.	Authentication in API layer and Token generation in Infrastructure.	Stateless authentication ideal for web hosting.
SignalR	Hubs, WebSockets, Long Polling fallback, `IHubContext`, User Connection mapping.	Real-time chat, typing indicators, read receipts, and live notification pushes.	Requires WebSockets enabled in hosting control panel.
BackgroundService (IHostedService)	Hosted worker lifecycle, `IServiceScopeFactory` for scoped DbContext resolution.	Birthday notification scheduler and recurring cleanup tasks.	Runs inside the IIS ASP.NET application pool.
SQL Server	Indexes, Foreign keys, cascade behaviors, query execution plans.	Relational database storage, schema management, and stored procedures if needed.	Hosted on ASP Monster dedicated MS SQL instance.
Frontend (HTML/CSS/JS/Tailwind)	DOM manipulation, Fetch API, SignalR JavaScript Client, WebSockets handling.	User profile UI, chat interface, dynamic feed rendering, notification drop-downs.	Static assets served via IIS static files middleware.

5. Phased Engineering Timeline & Delivery Process

The project development lifecycle is divided into **six structured execution phases**, guaranteeing orderly progress without tight date coupling:

Phase 1: Solution Architecture & Domain Foundation — **C# | Clean Architecture | Domain Layer**

- Initialize the solution structure with four distinct projects (API, Application, Infrastructure, Domain).
- Model pure Domain Entities: `AppUser`, `Post`, `Comment`, `PostLike`, `Friendship`, `Message`, `Notification`.
- Establish Enums: `FriendshipStatus`, `PostPrivacy`, `NotificationType`.
- Configure domain interfaces and base entity abstractions (`BaseEntity` with `CreatedAt`, `IsDeleted`).

Phase 2: Persistence Layer & Security Infrastructure

EF Core | SQL Server | JWT | Identity

- Implement `ApplicationDbContext` and configure Fluent API mappings (composite primary keys for friendships and likes).
- Implement the Generic Repository and `UnitOfWork` pattern inside the Infrastructure layer.
- Configure ASP.NET Core Identity with custom user properties (Bio, DateOfBirth, ProfilePictureUrl).
- Build `TokenService` to generate signed JWT tokens containing custom claims (UserId, DisplayName, Email).
- Run initial Code-First migrations and create the SQL Server database schema.

Phase 3: Core Social Engine & Search APIs

ASP.NET Core Web API | LINQ | DTOs

- Build Application Services and Controllers for Post creation, editing, deletion, and media uploading.
- Implement feed filtering logic based on `PostPrivacy` and friendship verification.
- Create Friendship Management endpoints (Send Request, Accept, Decline, Block, List Friends).
- Implement Instant Search service for users and posts with debounced backend query endpoints.
- Implement Likes and Comments interaction controllers.

Phase 4: Real-Time Chat & Notification Subsystems

SignalR | WebSockets | Application Services

- Create `ChatHub` with JWT authentication support via query-string access token parsing.
- Implement 1-on-1 private messaging logic with automatic sorting into "Inbox" or "Message Requests".
- Add real-time events for `UserTyping` indicators and `MessageRead` receipt updates.
- Build `NotificationHub` and event dispatchers for live notification toasts.
- Develop the `BirthdayNotificationWorker` background service utilizing `IServiceScopeFactory`.

Phase 5: User Interface Integration & SignalR Client

JavaScript | Tailwind / Bootstrap | SignalR Client

- Construct responsive UI views: Home Feed, Profile Pages, Chat Messenger, and Notification Center.
- Connect client-side JavaScript to SignalR Hubs for seamless message exchanges without page reloads.
- Build UI state indicators for unread message badges, friend request counters, and typing animations.
- Integrate media upload previews and dynamic feed search auto-complete.

Phase 6: Quality Assurance, Security Hardening & Monster ASP Hosting — IIS | Monster ASP / SmarterASP.NET | SQL Server Remote

- Perform end-to-end testing of business logic, private message routing, and privacy filters.
- Set up production `appsettings.Production.json` with remote connection strings and JWT signing secrets.
- Configure IIS `web.config`, static file caching, and WebSockets protocol support on the hosting panel.
- Deploy database migrations to the live Monster ASP SQL Server instance.
- Publish the compiled release build via FTP / Web Deploy and perform live validation.

6. Hosting & Deployment Guidelines (Monster ASP / SmarterASP.NET)

Key Deployment Checklist for Shared Windows Hosting:

- **WebSockets Activation:** Ensure the WebSockets feature is enabled in the hosting control panel so SignalR operates with minimal latency.
- **Application Pool:** Ensure the .NET CLR Version is set to "No Managed Code" to support .NET Core / .NET 8 in-process hosting.
- **Database Connection String:** Use the remote SQL Server IP, catalog name, and dedicated database credentials provided by the host.
- **File Storage:** Use relative paths mapped to `wwwroot/uploads` with appropriate IIS read/write permissions for user avatars and post images.